

# TALLER PRÁCTICO

## Programación Funcional en Java

*Lambdas · Interfaces funcionales · Streams · Referencias a métodos · Optional*

|                   |                                                   |
|-------------------|---------------------------------------------------|
| Curso             | Desarrollo de Aplicaciones Empresariales          |
| Docente           | Christian Merchán Millán                          |
| Modalidad         | Individual, en laboratorio o de forma remota      |
| Duración estimada | 30 minutos                                        |
| Tecnología        | Java 17+ (JDK), compilador javac / IDE a elección |

### 1. Introducción

La programación funcional es un paradigma que trata el cómputo como la evaluación de funciones, evitando el cambio de estado y los datos mutables. Desde Java 8, el lenguaje incorporó expresiones lambda, interfaces funcionales y la API de Streams, lo que permitió escribir código más declarativo, conciso y fácil de paralelizar.

En este taller el estudiante practicará, de forma guiada y paso a paso, los mecanismos fundamentales de la programación funcional en Java: expresiones lambda, interfaces funcionales estándar (Function, Predicate, Consumer, Supplier), referencias a métodos, la API Stream y el uso de Optional para evitar valores nulos.

### 2. Objetivos del taller

#### Objetivo general

Aplicar los principios de la programación funcional en Java mediante la construcción de un mini-programa que procese una lista de empleados utilizando lambdas y Streams.

#### Objetivos específicos

- 1) Escribir e invocar expresiones lambda utilizando interfaces funcionales propias y estándar de Java.
- 2) Aplicar operaciones intermedias y terminales de la API Stream (filter, map, sorted, reduce, collect).
- 3) Utilizar referencias a métodos (method references) como forma abreviada de lambdas.
- 4) Emplear la clase Optional para el manejo seguro de valores potencialmente nulos.
- 5) Identificar y corregir errores de compilación y de ejecución comunes en código funcional.

### 3. Requisitos del taller

## Conocimientos previos

- Sintaxis básica de Java: clases, métodos, colecciones (List, ArrayList).
- Concepto de interfaz e implementación de interfaces en Java.
- Manejo básico de un IDE (IntelliJ IDEA, Eclipse, VS Code) o de la terminal.

## Requisitos técnicos

- JDK 17 o superior instalado (recomendado; el taller funciona desde JDK 8 en adelante, salvo el uso de "var" y "record", opcional).
- Un editor de código o IDE con soporte para Java.
- Acceso a terminal o consola para compilar y ejecutar (javac y java), o un botón "Run" del IDE.

## Verificación previa

Antes de iniciar, confirme la versión de Java instalada ejecutando el siguiente comando en la terminal:

```
java -version
```

### Nota

Si el comando no es reconocido, instale el JDK desde <https://openjdk.org> (Open JDK) antes de continuar.

## 4. Resultados esperados

Al finalizar el taller, el estudiante habrá construido una clase Java funcional (TallerFuncional.java) capaz de:

- Filtrar una lista de empleados según su salario, usando un Predicate y/o Stream.filter.
- Transformar (mapear) los nombres de los empleados a mayúsculas usando Function y Stream.map.
- Ordenar la lista resultante por salario de forma descendente.
- Calcular el salario total y el promedio usando reduce y las utilidades de Collectors.
- Mostrar por consola un reporte final formateado.

La salida esperada en consola, con los datos de ejemplo del taller, es similar a la siguiente:

```
=== Empleados con salario > 1000 (ordenados desc.) ===
CARLOS  -> 2500.0
MARIA   -> 1800.0
PEDRO   -> 1200.0

Salario total del equipo filtrado: 5500.0
Salario promedio: 1833.33
Empleado mejor pagado: CARLOS
```

## 5. Desarrollo paso a paso

Cada paso indica una acción concreta, el código a escribir y el resultado que debe observar antes de continuar al siguiente paso.

### Paso 1 Crear el proyecto y la clase base

Cree una carpeta de trabajo llamada taller-funcional y, dentro de ella, un archivo llamado TallerFuncional.java. Escriba la estructura mínima de la clase con el método main:

```
import java.util.*;
import java.util.function.*;
import java.util.stream.*;

public class TallerFuncional {
    public static void main(String[] args) {
        System.out.println("Taller de Programación Funcional iniciado");
    }
}
```

Compile y ejecute para confirmar que el entorno funciona:

```
javac TallerFuncional.java
java TallerFuncional
```

#### Resultado esperado del paso 1

En consola debe aparecer: "Taller de Programación Funcional iniciado". Si esto no ocurre, revise la sección 6 (Errores comunes) antes de continuar.

### Paso 2 Definir el modelo Empleado y los datos de prueba

Dentro de la clase, agregue un record (o una clase POJO simple si usa JDK 8-15) para representar al empleado, y una lista de datos de prueba dentro de main:

```
// Java 17+: record inmutable, ideal para programación funcional
record Empleado(String nombre, double salario) {}

public static void main(String[] args) {
    List<Empleado> empleados = List.of(
        new Empleado("Carlos", 2500.0),
        new Empleado("Maria", 1800.0),
        new Empleado("Pedro", 1200.0),
        new Empleado("Ana", 900.0),
        new Empleado("Luis", 750.0)
    );
}
```

#### Alternativa JDK 8-15

Si su versión no soporta record, declare una clase tradicional con constructor, getNombre() y getSalario(), y ajuste las referencias a método del paso 4 (Empleado::getNombre en vez de Empleado::nombre).

### Paso 3 Filtrar con una expresión lambda (Predicate)

Declare un `Predicate<Empleado>` como una expresión lambda que evalúe si el salario supera 1000, y utilícelo dentro de un `Stream` para filtrar la lista:

```
Predicate<Empleado> salarioAlto = e -> e.salario() > 1000;

List<Empleado> filtrados = empleados.stream()
    .filter(salarioAlto)
    .collect(Collectors.toList());

System.out.println("Empleados con salario > 1000: " + filtrados.size());
```

### Resultado esperado del paso 3

El programa debe imprimir: "Empleados con salario > 1000: 3" (Carlos, Maria y Pedro).

## Paso 4 Transformar con Function y referencias a método

Use `Stream.map` junto con una referencia a método para convertir cada nombre a mayúsculas, y ordene el resultado por salario descendente con un `Comparator` basado en una referencia a método:

```
List<Empleado> ordenados = filtrados.stream()
    .sorted(Comparator.comparingDouble(Empleado::salario).reversed())
    .collect(Collectors.toList());

System.out.println("=== Empleados con salario > 1000 (ordenados desc.) ===");
ordenados.forEach(e ->
    System.out.printf("%-8s-> %.1f%n",
        e.nombre().toUpperCase(), e.salario())
);
```

Identifique en este bloque: (a) la expresión lambda usada en `forEach`, (b) la referencia a método `Empleado::salario` usada dentro de `comparingDouble`.

## Paso 5 Calcular totales con reduce y Collectors

Agregue el cálculo del salario total (usando `reduce`) y del promedio (usando `Collectors.averagingDouble`):

```
double total = ordenados.stream()
    .map(Empleado::salario)
    .reduce(0.0, Double::sum);

double promedio = ordenados.stream()
    .collect(Collectors.averagingDouble(Empleado::salario));

System.out.println();
System.out.println("Salario total del equipo filtrado: " + total);
System.out.printf("Salario promedio: %.2f%n", promedio);
```

## Paso 6 Manejar el valor máximo con Optional

Utilice `Stream.max`, que retorna un `Optional<Empleado>`, y resuelva ese valor de forma segura sin arriesgar un `NullPointerException`:

```
Optional<Empleado> mejorPagado = ordenados.stream()
    .max(Comparator.comparingDouble(Empleado::salario));

mejorPagado.ifPresentOrElse(
    e -> System.out.println("Empleado mejor pagado: " +
    e.nombre().toUpperCase()),
    () -> System.out.println("No hay empleados que cumplan el criterio")
);
```

### Resultado esperado del paso 6

"Empleado mejor pagado: CARLOS". Compile y ejecute todo el archivo: la salida completa debe coincidir con la mostrada en la sección 4 (Resultados esperados).

## Paso 7 Verificación final y entrega

Antes de dar por concluido el taller, revise la siguiente lista de verificación:

- ☐ El archivo compila sin errores (javac TallerFuncional.java).
- ☐ La ejecución (java TallerFuncional) produce la salida esperada de la sección 4.
- ☐ El código usa al menos una expresión lambda, una referencia a método, un Stream y un Optional.
- ☐ El archivo .java está guardado con el nombre exacto de la clase pública.

Guarde el archivo final y espere las indicaciones del profesor para la sesión de revisión en clase.

## 6. Posibles errores y cómo resolverlos

La siguiente tabla resume los errores más frecuentes al realizar este taller, agrupados por el paso donde suelen aparecer.

| Error / Síntoma                                                                                        | Causa probable                                                                                                         | Cómo resolverlo                                                                                                     |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>error: class TallerFuncional is public, should be declared in a file named TallerFuncional.java</b> | El nombre del archivo no coincide exactamente (mayúsculas/minúsculas incluidas) con el nombre de la clase pública.     | Renombre el archivo para que coincida exactamente con el nombre de la clase, incluyendo mayúsculas.                 |
| <b>error: incompatible types: bad return type in lambda expression</b>                                 | El cuerpo de la lambda no retorna el tipo esperado por la interfaz funcional (p. ej. Predicate espera boolean).        | Revise la firma del método abstracto de la interfaz funcional y ajuste el tipo de retorno de la lambda.             |
| <b>error: local variables referenced from a lambda expression must be final or effectively final</b>   | Se intenta reasignar dentro de la lambda una variable local externa (por ejemplo, un contador con i++).                | Use una variable efectivamente final, o reemplace el conteo/acumulación por reduce, sum() o un Collectors adecuado. |
| <b>NullPointerException al invocar un método sobre el resultado de max(), min() o findFirst()</b>      | Se llamó a .get() directamente sobre el Optional sin verificar si contiene un valor (lista vacía o sin coincidencias). | Use ifPresent, ifPresentOrElse, orElse u orElseThrow en lugar de .get() sin verificación.                           |
| <b>error: cannot find symbol — método salario() o nombre()</b>                                         | Se está usando la sintaxis de record (Java 17+) con un JDK anterior, o se declaró una clase tradicional pero se        | Confirme la versión de JDK con java -version; si es menor a 17, use una                                             |

| Error / Síntoma                                                                                          | Causa probable                                                                                         | Cómo resolverlo                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                          | llamó a métodos de accesor de record.                                                                  | clase POJO con <code>getSalario()/getNombre()</code> .                                                                                                                 |
| <b>UnsupportedOperationException al intentar agregar o eliminar elementos de la lista</b>                | List.of(...) devuelve una lista inmutable; el código intenta modificarla (add, remove, sort in-place). | No modifique la lista original; use streams para producir nuevas colecciones, o envuélvala en <code>new ArrayList&lt;&gt;(lista)</code> si de verdad necesita mutarla. |
| <b>El stream ya fue operado — IllegalStateException: stream has already been operated upon or closed</b> | Se intenta reutilizar la misma variable de tipo Stream en más de una operación terminal.               | Genere un nuevo stream ( <code>lista.stream()</code> ) por cada cadena de operaciones; los streams son de un solo uso.                                                 |

## 7. Reto opcional (extensión, no obligatorio)

Si terminó el taller antes del tiempo estimado, puede seguir trabajando con una de las siguientes variaciones:

- Agrupar los empleados por rango salarial usando `Collectors.groupingBy`.
- Implementar una interfaz funcional propia (por ejemplo, `CalculadoraBono`) con un método abstracto y usarla con una lambda.
- Convertir el reporte final a formato de texto usando `Collectors.joining` en lugar de `forEach`.

## 8. Cierre y autoevaluación

Responda brevemente, en sus propias palabras, antes de entregar el taller:

- 1) ¿Qué ventaja ofrece usar Stream frente a un bucle for tradicional en este ejercicio?
- 2) ¿Por qué Optional evita un error en tiempo de ejecución al buscar el empleado mejor pagado?
- 3) ¿Qué diferencia existe entre una expresión lambda y una referencia a método?